

Python 核心特性复习手册

从语言语义到 CPython、并发、工程实践与性能优化

适合已有编程经验、希望系统复习Python 的工程师



Python 核心特性复习指南

简洁 · 优雅 · 强大 · 生态丰富



易读易学

语法简洁
学习曲线平缓



动态类型

解释执行
灵活高效



多范式

面向过程
面向对象
函数式编程



丰富标准库

开箱即用
功能强大



跨平台

Windows / Linux
macOS



庞大生态

PyPI 生态丰富
社区活跃

目录

1. 语言基础
2. 数据结构
3. 函数与作用域
4. 面向对象编程
5. 异常处理
6. 模块与包
7. 迭代器与生成器
8. 装饰器与上下文管理器
9. 并发编程
10. 常用标准库
11. 文件与 I/O
12. 测试与调试
13. 性能优化
14. 最佳实践
15. 常见陷阱

1. 语言基础

1.1 变量与基本类型

- 动态类型, 无需声明类型
- 一切皆对象

```
x = 10          # int
y = 3.14        # float
s = "hello"     # str
b = True        # bool
n = None        # NoneType
```

1.3 控制流程

- 缩进表示代码块

```
if x > 0:
    print("positive")
elif x == 0:
    print("zero")
else:
    print("negative")

for i in range(3):
    print(i)
```

1.2 运算符

- 算术, 比较, 逻辑, 位运算等

```
a = 10; b = 3
a + b    # 13
a // b   # 3
a % b    # 1
a * b    # 30
a > b and b < 5  # True
```

1.4 推导式

- 列表、字典、集合、生成器推导式

```
[i*i for i in range(5)]
{x: x*x for x in range(3)}
{x for x in range(5) if x%2==0}
(i*i for i in range(5))  # 生成器
```

2. 数据结构

2.1 列表 (List)

- 有序、可变、可重复

```
lst = [1, 2, 3]
lst.append(4)
lst[1] = 10
lst[1:3]  # 切片
```

2.3 字典 (Dict)

- 键值对, 无序 (Python 3.7+ 保持插入顺序)

```
d = {"a": 1, "b": 2}
d["c"] = 3
d["a"]  # 1
d.get("a", 0)  # 0
```

2.5 字符串 (Str)

- 不可变的字符串序列

```
s = "hello python"
s[0]  # 'h'
s[1:5]  # 'ello'
s.upper()  # 'HELLO PYTHON'
"py" in s  # True
```

2.2 元组 (Tuple)

- 有序、不可变、可重复

```
t = (1, 2, 3)
x, y, z = t  # 解包
t[0]  # 1
```

2.4 集合 (Set)

- 无序、唯一

```
s = {1, 2, 3}
s.add(4)
s.remove(2)
```

2.6 常用操作函数

```
len(x)          # 长度
min(x) / max(x) # 最小/最大值
sorted(x)        # 排序
sum(x)           # 求和
enumerate(x)     # 枚举索引
zip(x, y)        # 合并迭代器
```

3. 函数与作用域

3.1 函数定义

```
def add(a, b=0, *args, **kwargs):
    """示例函数"""
    return a + b
```

3.2 作用域规则 (LEGB)

Local → Enclosing → Global → Built-in

3.3 匿名函数

```
f = lambda x: x * x
list(map(lambda x: x+1, [1,2,3]))
```

3.4 类型注解 (PEP 484)

```
from typing import List, Optional

def f(name: str, age: Optional[int]) -> List[str]:
    ...
```

4. 面向对象编程

4.1 类与对象

```
class Person:
    def __init__(self, name):
        self.name = name

    def say(self):
        return f"Hello {self.name}"

p = Person("Alice")
p.say()  # Hello Alice
```

4.2 继承与多态

```
class Student(Person):
    def __init__(self, name, school):
        super().__init__(name)
        self.school = school

    def say(self):
        return f"Hi, I'm {self.name}"

# 多态
people = [Person("A"), Student("B", "PKU")]
for p in people:
    print(p.say())
```

4.3 特殊方法

• `__init__` 初始化 • `__len__` 长度
• `__str__` 字符串表示 • `__eq__` 比较
• `__repr__` 调试表示 • `__iter__` 迭代

5. 异常处理

5.1 try / except / finally

```
try:
    x = 1 / 0
except ZeroDivisionError as e:
    print("error:", e)
else:
    print("no error")
finally:
    print("finally")
```

5.2 自定义异常

```
class MyError(Exception):
    pass

raise MyError("something wrong")
```

常见内置异常

• `ZeroDivisionError` • `IndexError`
• `IndexError` • `KeyError`
• `TypeError` • `FileNotFoundError`
• `RuntimeError` • `Exception (基类)`

6. 模块与包

6.1 导入模块

```
import math
import os.path as osp
from collections import deque
from mypkg.mymodule import func
```

6.2 包结构

```
mypackage/
├── __init__.py
├── module1.py
├── module2.py
├── subpkg/
│   ├── __init__.py
│   └── module3.py
```

6.3 `__name__` 与 `__main__`

```
if __name__ == "__main__":
    main()
```

包用于组织代码, `__init__.py` 表示目录是包
相对导入: `from . import module1`

7. 迭代器与生成器

7.1 迭代器协议

- 对象实现 `__iter__()` 和 `__next__()` 方法

7.2 生成器函数

```
def count(n):
    i = 0
    while i < n:
        yield i
        i += 1

for x in count(3):
    print(x)
```

7.3 生成器表达式

```
{x*x for x in range(5)}  # 生成器
sum(x*x for x in range(5))
```

- 生成器惰性计算, 节省内存
- 可与 `next()` 函数配合使用

8. 装饰器与上下文管理器

8.1 装饰器 (Decorator)

```
def log(func):
    def wrapper(*args, **kwargs):
        print(f"call {func.__name__}")
        return func(*args, **kwargs)
    return wrapper

@log
def add(a, b):
    return a + b
```

8.2 上下文管理器 (Context Manager)

```
with open("file.txt", "r") as f:
    data = f.read()

# 自定义上下文管理器
class MyCtx:
    def __enter__(self):
        print("enter")
    def __exit__(self, exc_type, exc, tb):
        print("exit")
```

9. 并发编程

9.1 多线程 (Threading)

```
import threading
def worker():
    print("working")

t = threading.Thread(target=worker)
t.start()
t.join()
```

9.2 线程同步

```
from threading import Lock
lock = Lock()

with lock:
    # 临界区
```

9.3 多进程 (Multiprocessing)

```
from multiprocessing import Process
def worker():
    print("process")

if __name__ == "__main__":
    p = Process(target=worker)
    p.start()
    p.join()
```

线程 vs 进程			
特性	线程	进程	备注
内存共享	是	否	通过引用实现
启动速度	快	慢	大
资源消耗	小	大	高内存/高IO
管理/协作	简单	复杂	管理/协作/共享内存

10. 常用标准库

`os / os.path` 系统接口与路径操作
`sys` 解释器参数与环境
`re` 正则表达式
`collections` 扩展数据结构 (deque, Counter 等)
`itertools` 高效迭代器工具
`functools` 高阶函数与工具 (lru_cache 等)
`datetime` 日期与时间
`json` JSON 编解码
`logging` 日志
`argparse` 命令行解析
`typing` 类型提示支持
`pathlib` 面向对象的路径操作
`random` 随机数生成

11. 文件与 I/O

11.1 文件读写

```
with open("data.txt", "r", encoding='utf-8') as f:
    text = f.read()

with open("out.txt", "w", encoding='utf-8') as f:
    f.write(text)
```

11.2 二进制文件

```
with open('img.png', 'rb') as f:
    data = f.read()

with open('copy.png', 'wb') as f:
    f.write(data)
```

11.3 序列化

```
import json, pickle

json.dumps({'a': 1})
pickle.dumps([1,2,3])
```

12. 测试与调试

12.1 单元测试 (unittest)

```
import unittest

class TestAdd(unittest.TestCase):
    def test_add(self):
        self.assertEqual(1+2, 3)

if __name__ == "__main__":
    unittest.main()
```

12.2 断言 (assert)

```
x = 10
assert x == 10, "x must be 10"
```

12.3 调试 (pdb)

```
$ python -m pdb script.py
(Pdb) list
(Pdb) n
(Pdb) v variable
```

13. 性能优化

13.1 常用技巧

- 使用局部变量 (查找快)
- 会使用内置函数和函数 (如 list/dict/set)
- 使用生成器避免大内存占用
- 避免在循环中重复计算
- 使用 `lru_cache` 缓存计算结果

13.2 性能分析工具

• `cProfile`
• `line_profiler`
• `memory_profiler`

14. 最佳实践 (PEP 8)

- 使用有意义的命名 (变量、函数、类)
- 遵循 PEP 8 编码规范
- 编写函数要短小、单一职责
- 避免全局变量
- 用异常处理代替返回错误码
- 编写文档字符串 (Docstring)
- 使用虚拟环境管理项目依赖

```
def add(a, b):
    """Return the sum of a and b."""
    Args:
        a (int): first number
        b (int): second number
    Returns:
        int: sum of a and b
    """
    return a + b
```

15. 常见陷阱

- 可变对象作为默认参数
- `def f(x, l=[]): ...` # 错误
- late binding 在循环中的 lambda
- `funcs = [lambda x: i for i in range(3)]` # i 都是最后一个值
- 浮点数精度问题
- `0.1 + 0.2 != 0.3`
- 使用 `is` 比较不可变对象需谨慎
- 检测列表/字典的遍历
- 忘记关闭资源或异常未处理

阅读方法 第一次快速浏览目录和粗体结论；第二次亲手运行代码；第三次重点复习对象模型、GIL/asyncio、生成器、typing、GC 与性能分析。

1. Python 的核心设计与执行模型

Python 的重要特征并不只是“语法简单”。真正影响工程行为的是动态类型、对象模型、引用语义、解释器执行模型、强大的协议机制以及庞大的标准库和生态。

特性	含义	工程影响
动态类型	变量名绑定对象，类型属于对象而不是变量	快，但需要、typing 和清晰接口
一切皆对象	函数、类、模块、类型本身都是对象	支持高阶函数、装饰器、元编程
引用语义	赋值通常复制引用，不复制对象	可变对象共享是高频 bug 来源
协议优先	迭代、上下文管理、数字运算等由特殊方法协议驱动	duck typing 与组合非常自然
解释器生态	CPython 是主流实现，但语言规范不等于 CPython	理解 GIL/GC 时要区分语言与实现

1.1 从源码到执行

```
source.py
  ↓ tokenize / parse
AST
  ↓ compile()
code object / bytecode
  ↓ CPython evaluation loop
Python objects + C runtime + OS
```

CPython 把源成 code object，其中包含字节码、常量表、变量名等。随后由解释器执行字节码。现代 CPython 做自适解指令化，但角度仍可把理解“成字，再由机行”。

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
```

关键区别 Python 语言规范定义语义；CPython 决定大量实现细节。GIL、引用计数、对象布局、某些性能特征主要属于 CPython。

2. 名字绑定、对象、可变性与身份

2.1 变量不是盒子，而是名字

```
a = [1, 2]
b = a
b.append(3)
print(a) # [1, 2, 3]
print(a is b) # True
```

这里 a 和 b 指向同一个 list 对象。理解“名字 → 对象”的绑定关系，是理解参数传递、闭包、默认参数、浅拷贝和并发共享状态的基础。

概念	操作	结论
identity	a is b	是否为同一个对象
equality	a == b	值是否相等，由 __eq__ 决定
mutation	obj.append(...)	对象本身变化
rebinding	x = new_obj	名字改为指向新对象

2.2 不可变与可变对象

- 常见不可变：int、float、bool、str、bytes、tuple（前提是内部元素本身可哈希/不影响 tuple 身份语义）。
- 常见可变：list、dict、set、bytearray、自定义对象。
- 可哈希对象才能作为 dict key 或 set 元素；通常要求 hash 值在生命周期中稳定。

```
x = "ab"
y = x
x += "c" # 新建字符串，再把 x 重新绑定

lst = [1, 2]
lst += [3] # 原地扩展 list
```

复习重点 看到赋值、参数传递或闭包时，先问：这是 mutation 还是 rebinding？对象是 mutable 还是 immutable？

3. 核心数据结构：list / tuple / dict / set / str

类型	主要特性	典型复杂度	适合
list	动态数组，有序，可变	索引 $O(1)$ ，尾部 append 摊销 $O(1)$	序集合、
tuple	有序，不可变	索引 $O(1)$	固定记录、可哈希组合键
dict	哈希表，保持插入顺序	平均查找/插入 $O(1)$	映射、存、索引
set	哈希集合，无重复	平均成员测试 $O(1)$	去重、集合运算
str	Unicode 文本，不可变	切片创建新对象	文本处理

3.1 list：动态数组与切片

```
a = [1, 2, 3, 4]
b = a[1:3]    # 新 list
a[1:3] = [9, 8] # 原 list 被修改

stack = []
stack.append(1)
stack.pop()
```

list 适合随机访问和尾部追加，不适合频繁从头部插入/除；列通常使用 `collections.deque`。

3.2 dict：哈希、顺序与键

```
d = {"name": "alice", "age": 18}
for k, v in d.items():
    print(k, v)

value = d.get("missing", 0)
```

- dict 自 Python 3.7 起语言层面保证插入顺序。
- key 必须可哈希；list/dict/set 不能直接作为 key。
- 成员测试 `x in d` 检查 key，而不是 value。
- defaultdict / Counter / OrderedDict / ChainMap 在特定模式下更清晰。

3.3 字符串与 bytes

```
text = "你好"
data = text.encode("utf-8")
assert data.decode("utf-8") == text
```

工程原则 文本边界上明确区分 str 与 bytes。网络、文件、压缩、加密等系统接口经常以 bytes 工作。

4. 函数、参数模型、作用域与闭包

4.1 函数是一等对象

```
def apply(fn, x):
    return fn(x)

def square(x):
    return x * x

print(apply(square, 4))
```

- 函数可赋值给变量、传参、返回、放入容器。
- 函数对象携带 `\_\_name\_\_`、`\_\_doc\_\_`、`annotations`、`closure` 等元信息。

4.2 参数传递：call by sharing

```
def mutate(xs):
    xs.append(1)

def rebind(xs):
    xs = [99]

a = []
mutate(a) # a -> [1]
rebind(a) # a 仍然 [1]
```

Python 的是象引用的共享。函数可以修改共享的变量，但重新定局部名不改用者名字的绑定。

4.3 *args / **kwargs 与关键字参数

```
def connect(host, port=443, *, timeout=3, **options):
    ...

connect("example.com", timeout=5, retries=2)
```

- `/` 可声明 positional-only 参数；`\*` 之后是 keyword-only 参数。
- API 设计中 keyword-only 参数有助于提高可读性并减少调用歧义。

4.4 LEGB、global、nonlocal 与闭包

```
def counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
```

```
return n  
return inc
```

闭包陷阱 循环变量是 late binding。需要捕获当前值时，可使用默认参数或 `functools.partial`。

5. 面向对象：类、MRO、dataclass 与协议

Python OOP 的重点不是传统“深继承树”，而是对象协议、组合、duck typing、mixins、dataclass 以及动态能力。

```
from dataclasses import dataclass

@dataclass(frozen=True, slots=True)
class User:
    id: int
    name: str
```

5.1 类与实例

- 实例属性通常存储在 `\_\_dict\_\_`，使用 `\_\_slots\_\_` 可限制属性并降低部分对象开销。
- 方法访问本质是 descriptor binding：`obj.method` 会把函数绑定为携带 self 的 bound method。
- classmethod 接收 cls；staticmethod 不做绑定。

5.2 继承与 MRO

```
class A: ...
class B(A): ...
class C(A): ...
class D(B, C): ...

print(D.__mro__)
```

多承使用 C3 linearization 计算 MRO。合作式多继承要求各层正确使用 `super()`，而不是直接调用父类名。

5.3 property、descriptor 与魔术方法

- `@property` 把方法暴露成属性访问接口。
- descriptor 协议 `\_\_get\_\_` / `\_\_set\_\_` / `\_\_delete\_\_` 是 property、method、很多 ORM 字段的基础。
- 常用协议：`\_\_iter\_\_`、`\_\_len\_\_`、`\_\_enter\_\_`、`\_\_call\_\_`、`\_\_eq\_\_`、`\_\_hash\_\_`、`\_\_getattr\_\_`。

设计建议 优先组合和小接口。需要“像某种对象一样工作”时，思考协议与 Protocol，而不是强制继承某个基类。

6. 迭代器、生成器与惰性计算

6.1 Iterable 与 Iterator

```
it = iter([10, 20, 30])
print(next(it))
print(next(it))
```

- Iterable : 能产生 iterator , 通常实现 `\_\_iter\_\_()` 。
- Iterator : 实现 `\_\_iter\_\_()` 和 `\_\_next\_\_()` ; 耗尽后抛 StopIteration 。

6.2 generator

```
def read_chunks(f, size=8192):
    while chunk := f.read(size):
        yield chunk
```

yield 会把函数变成生成器函数。调用时不立即执行函数体，而是返回 generator 对象；每次 next() 恢复执行直到下一个 yield。适合流式处理和降低峰值内存。

6.3 生成器表达式

```
total = sum(x * x for x in range(1_000_000))
```

6.4 yield from

```
def flatten(groups):
    for g in groups:
        yield from g
```

思维模型 list comprehension = 立即构建结果；generator expression = 惰性拉取。先确认是否需要随机访问/重复遍历，再决定是否物化。

7. Decorator 、Context Manager 与异常处理

7.1 Decorator

```
from functools import wraps

def trace(fn):
    @wraps(fn)
    def wrapper(*args, **kwargs):
        print("call", fn.__name__)
        return fn(*args, **kwargs)
    return wrapper
```

装饰器本质上是“接收 callable，返回 callable”的高阶函数。`functools.wraps` 用于保留被包装函数的元信息。

7.2 Context Manager

```
with open("data.txt", encoding="utf-8") as f:
    text = f.read()
```

- 协议：`\_\_enter\_\_()` / `\_\_exit\_\_()`。
- 适合资源生命周期：文件、锁、数据库事务、临时目录、trace span。
- `contextlib.contextmanager` 可以用 generator 编写上下文管理器。

7.3 Exceptions

```
try:
    value = int(text)
except ValueError as e:
    raise ConfigError("invalid port") from e
else:
    use(value)
finally:
    cleanup()
```

- 捕获尽可能具体的异常；避免裸 `except:`。
- 用 `raise ... from e` 保留异常链。
- 异常适合真正异常路径，不建议作为高频控制流。
- 自定义异常通常继承 `Exception`，并保持层次简单。

8. 模块、包、导入系统与虚拟环境

```
project/  
  pyproject.toml  
  src/myapp/  
    __init__.py  
    api.py  
    service.py  
  tests/
```

- module 基本对应一个 .py 文件；package 是可包含子模块的命名空间。
- import 首先查看 `sys.modules` 缓存，再经过 finder / loader 机制解析。
- `if \_\_name\_\_ == "\_\_main\_\_":` 区分模块被导入还是作为脚本执行。
- 推荐使用虚拟环境隔离依赖；现代项目优先以 `pyproject.toml` 管理构建元数据。

8.1 import 的副作用

模块首次 import 时顶层代码会执行。因此应避免在 import 时进行昂贵网络调用、启动线程、连接数据库或修改全局状态。

8.2 循环导入

- 常见根因是模块职责边界不清。
- 优先重构共享类型到更低层模块，而不是到处使用函数内部 import。
- 必要时可以局部 import，但它更像应急手段。

9. Typing：从动态语言到可检查接口

类型注解不会默认在运行时强制检查，但它能提升 IDE、静态分析、重构和大型团队协作质量。

```
from collections.abc import Iterable
from typing import Protocol, TypeVar

class SupportsClose(Protocol):
    def close(self) -> None: ...

T = TypeVar("T")
def first(items: Iterable[T]) -> T:
    return next(iter(items))
```

- 优先使用内建泛型：`list[str]`、`dict[str, int]`。
- `X | None` 表达 Optional。
- Protocol 支持结构化子类型，更贴合 duck typing。
- TypedDict 用于描述 dict 形状；Literal 可限制字面值；TypeAlias/Annotated 在大型 API 中很有用。
- 类型注解应服务于边界与可维护性，不应为了“100% 类型体操”牺牲可读性。

9.1 Pattern Matching

```
match event:
    case {"type": "created", "id": item_id}:
        handle_create(item_id)
    case {"type": "deleted", "id": item_id}:
        handle_delete(item_id)
    case _:
        ignore()
```

10. 并发模型总览：threading / multiprocessing / asyncio

模型	并行能力	适合	主要成本/限制
threading	CPython CPU 密集通常受 GIL 限制	I/O 密集、阻塞库	线程同步、共享状态
multiprocessing	真正多核并行	CPU 密集	程建、IPC、序列化
asyncio	单线程事件循环为主，协作式并发	大量网络 I/O、高连接数	必须使用 async-friendly 调用链

10.1 GIL 是什么

在主流 CPython 中，GIL（Global Interpreter Lock）限制同一解释器进程内多个线程同时执行 Python 字节码。它简化了 CPython 对象模型和引用计数的同步，但使纯 Python CPU 密集线程难以线性利用多核。

- GIL 不等于“Python 没有并发”。I/O 操作和很多 C 扩展不受 GIL 限制。
- I/O 密集任务 threading 仍可能获得很好吞吐。
- CPU 密集可考虑 multiprocessing、原生扩展、NumPy 等。
- 不要把线程安全等同于 GIL 安全；复合操作仍可能需要 Lock。

```
from concurrent.futures import ThreadPoolExecutor

with ThreadPoolExecutor(max_workers=20) as ex:
    results = list(ex.map(fetch, urls))
```

11. asyncio：事件循环、Task、Future 与取消

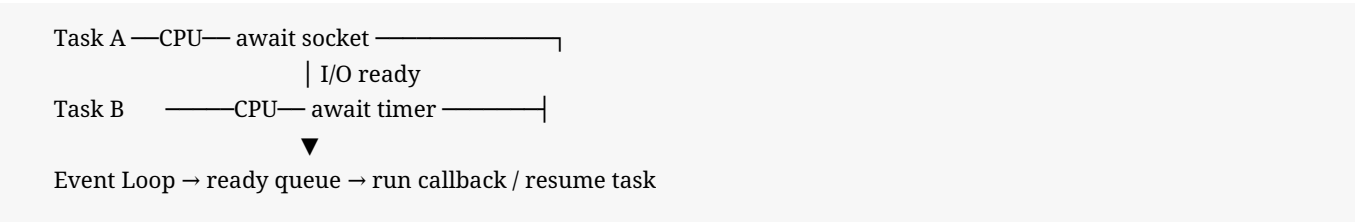
```
import asyncio

async def fetch_one(client, url):
    async with client.get(url) as resp:
        return await resp.text()

async def main():
    tasks = [asyncio.create_task(work(i)) for i in range(100)]
    return await asyncio.gather(*tasks)

asyncio.run(main())
```

asyncio 是协作式并发：一个 Task 运行到 await 处主动让出控制权，事件循环再运行其他就绪 Task。当对应 I/O 完成时，Task 被重新放回可运行集合。



11.1 关键对象

对象	作用
coroutine	调用 async def 后得到的可等待对象
Task	调度 coroutine 的运行实体
Future	未来某个时刻可完成的结果占位符
Event Loop	处理就绪任务、I/O 事件、定时器和 callbacks

11.2 Structured Concurrency

```
async with asyncio.TaskGroup() as tg:
    for item in items:
        tg.create_task(process(item))
```

- TaskGroup 让子任务生命周期有结构地绑定到父作用域。
- 取消是 async 系统的重要控制流；代码应正确传播 CanceledError。
- 不要在 event loop 中直接运行长时间阻塞函数；使用异步库或 executor。

12. 内存管理：引用计数、循环 GC 与 weakref

CPython 主要依靠引用计数管理对象生命周期，并辅以循环垃圾回收器处理引用环。

```
name └─┐
      │→ object  refcount = 2
alias └─┘

del name → refcount = 1
del alias → refcount = 0 → deallocate
```

- 引用计数使很多对象在引用数归零时立即释放，这是 CPython 的典型行为。
- 循环引用无法仅靠引用计数释放，因此 cyclic GC 会扫描容器对象图。
- ``gc`` 模块可查看/控制循环 GC；``weakref`` 可避免缓存或观察者关系形成强引用。
- ``__del__`` 与复杂资源清理容易产生生命周期问题；优先使用 context manager。

12.1 对象大小与复制

- ``sys.getsizeof()`` 只返回对象本身的浅层大小，不包含递归引用对象。
- ``copy.copy()`` 是浅拷贝；``copy.deepcopy()`` 递归复制，成本更高且未必符合业务语义。
- 大对象链路中要关注临时对象、字符串拼接、中间 list 物化。

13. 性能：先测量，再优化

工具	用途
timeit	微基准
cProfile / pstats	函数级 CPU profiling
tracemalloc	内存分配追踪
py-spy	低侵入采样 profiler
line_profiler	行级热点分析
memory_profiler	内存变化观察

13.1 常见优化方向

- 算法和数据结构优先： $O(n^2) \rightarrow O(n \log n)$ 通常远胜于微优化。
- 减少 Python 层循环：使用内建函数、标准库、NumPy/向量化。
- 避免重复解析、序列化、正则编译、数据库往返。
- 缓存要先定义失效策略和容量边界。
- I/O 系统关注批量、连接池、背压和并发控制。
- 热点 CPU 逻辑可考虑 Cython、Rust/C 扩展、NumPy 或多进程。

```
python -m cProfile -o app.prof app.py
python -m timeit "sum(range(1000))"
```

原则 不要根据“感觉”优化。先 profile 找热点，再对热点做可验证的改动，并用 benchmark 防止回归。

14. 测试、调试与工程工具链

14.1 测试分层

层级	目标	典型工具
Unit	快速验证纯逻辑	pytest / unittest
Integration	验证数据库、队列、外部服务契约	pytest + containers/mocks
E2E	验证真实用户链路	API/UI test
Property-based	探索边界输入和不变量	Hypothesis

```
def test_parse_port():
    assert parse_port("8080") == 8080

import pytest
with pytest.raises(ValueError):
    parse_port("bad")
```

14.2 常用工具

- ruff : lint + 一部分格式/导入能力。
- black : 格式化。
- mypy / pyright : 静态类型检查。
- pytest : 测试框架和丰富插件生态。
- coverage.py : 覆盖率。
- pdb / breakpoint() : 交互调试。
- tox / nox : 多环境测试和自动化。

14.3 Packaging

- 现代项目核心元数据放在 `pyproject.toml` 。
- 区分 application 与 library 的依赖管理策略。
- 锁文件用于应用可重复部署；库通常避免过度锁死下游依赖。

15. 标准库高频模块地图

领域	模块
路径/文件	pathlib, os, shutil, tempfile
数据结构	collections, heapq, bisect, array
函数式	itertools, functools, operator
时间	datetime, zoneinfo, time
序列化	json, csv, pickle（仅可信数据）
并发	threading, multiprocessing, concurrent.futures, asyncio
网络	socket, http, urllib
日志	logging
进程	subprocess
文本/匹配	re, string, textwrap
类型/对象	typing, dataclasses, enum
测试/调试	unittest, doctest, pdb, tracemalloc

安全提醒 不要对不可信输入使用 pickle.load。需要跨服务交换数据时优先 JSON、MessagePack、Protobuf 等显式格式。

16. 常见陷阱与高频面试题

陷阱	错误示例/现象	正确理解
可变默认参数	<code>def f(xs=[]): ...</code>	默认值只在函数定义时创建一次，用 <code>None</code> 作为哨兵
浅拷贝	<code>[[0]*3]*3</code>	三个元素可能引用同一个子 list
<code>is</code> vs <code>==</code>	<code>x is 1000</code>	<code>is</code> 检查身份，不用于普通值比较
闭包 late binding	<code>lambda in loop</code>	闭包读取变量当前值，不自动快照
dict 修改期间迭代	<code>for k in d: del d[k]</code>	先复制 keys 或构造新 dict
async 阻塞	<code>time.sleep()</code> in <code>async def</code>	用 <code>asyncio.sleep</code> 或 <code>executor</code>
吞异常	<code>except Exception: pass</code>	记录上下文并只捕获可处理异常
线程即并行	CPU-heavy threading	CPython 受 GIL 影响，考虑进程/原生扩展
导入副作用	import 时启动服务	保持模块 import 轻量
深继承	复杂 MRO	优先组合与 Protocol

16.1 自测问题

1. 为什么 ``a=b=[]`` 修改 `b` 会影响 `a`？
2. tuple 一定可哈希吗？
3. 为什么 dict key 要求可哈希？
4. Python 参数传递到底是值传递还是引用传递？
5. 闭包中的 `nonlocal` 与 `global` 有什么区别？
6. `classmethod` 和 `staticmethod` 的绑定行为有何不同？
7. generator 为什么可以降低内存峰值？
8. GIL 为什么不等于线程完全没用？
9. `asyncio` 中 `await` 发生时事件循环做了什么？
10. 引用计数为什么不能独立解决循环引用？
11. Protocol 为什么比强制继承更贴合 duck typing？
12. 如何定位一个 Python 服务的 CPU 热点和内存增长？

17.7 天复习路线

Day	主题	目标
Day 1	对象模型 + 可变性 + list/dict/set	能解释引用、拷贝、哈希、身份与相等
Day 2	函数 + 闭包 + decorator + context manager	掌握 Python 的函数式与协议式表达
Day 3	OOP + dataclass + MRO + Protocol	能设计可维护的对象接口
Day 4	iterator/generator + typing + package	掌握现代 Python API 与代码组织
Day 5	threading + multiprocessing + GIL	能为 CPU/I/O 工作负载选正确并发模型
Day 6	asyncio + event loop + cancellation	能读懂和设计 async 服务
Day 7	GC + profiling + testing + pitfalls	能定位问题并完成一次综合复盘

17.1 最后应该能回答的四个问题

- Python 代码从 .py 到运行时对象经历了什么？
- 对象共享、可变性和作用域如何影响程序正确性？
- threading / multiprocessing / asyncio 分别解决什么问题？
- 出现 CPU 高、内存涨、延迟高时，应该如何用工具定位而不是猜？

建议练习 用 FastAPI 或标准库 asyncio 写一个小型并发服务：包含 dataclass/typing、async I/O、线程池接阻塞函、 pytest、logging 和 profiling。一次项目能覆盖大部分关键知识。